

EE 446: TinyML
Lab 7: TinyML for Motion Classification
On Device Data Collection & Ensemble Learning

Kourosh
Student ID: 2331288
kourosh@uw.edu

Summer 2026

Part I: On Device Data Collection and Motion Classification

Edge Impulse Project Link

The public Edge Impulse project for this lab is available at:

https://studio.edgeimpulse.com/public/PROJECT_ID/live

Data Collection

The Arduino Nano 33 BLE Sense Rev2 was connected to Edge Impulse using the Edge Impulse CLI serial daemon. The Edge Impulse firmware was flashed onto the board first, since the stock firmware does not expose the remote management interface that Edge Impulse Studio uses to talk to the device. Once the daemon was running, the board appeared under the Devices tab and became selectable in the Data Acquisition panel.

Two classes were recorded. The Idle class was collected with the board resting flat and stationary on a desk. The Lift class was collected with the board picked up and held in the air, which changes both the orientation of the gravity vector across the accelerometer axes and the amount of small-scale motion present in the signal. Twenty samples were recorded per class, each ten seconds long, using the Inertial and Magnetometer sensor at 100 Hz. This produced forty samples and six minutes and forty seconds of total data.

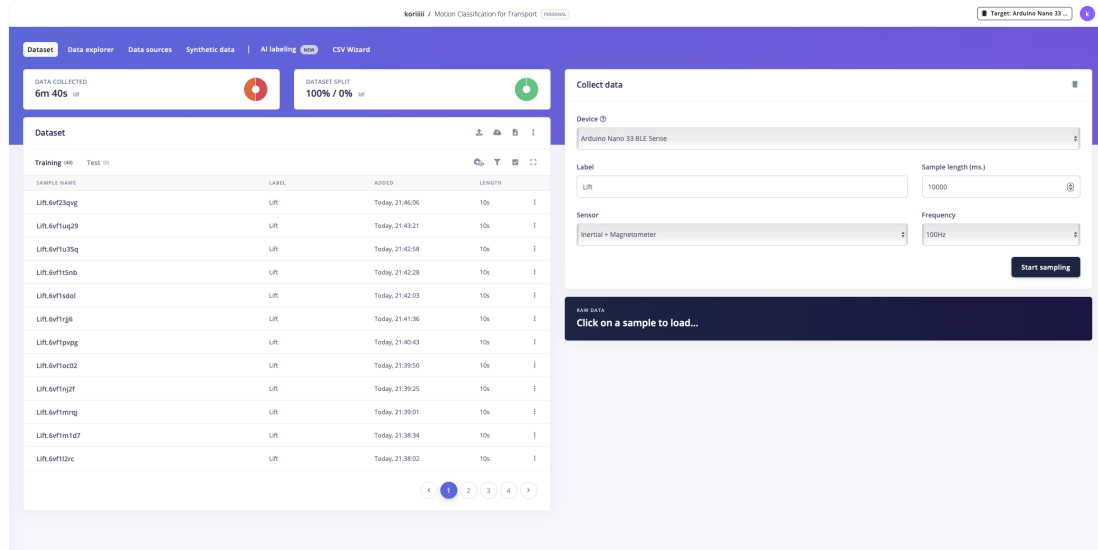


Figure 1: Data acquisition panel after collecting all forty samples

The dataset was then split into training and test sets using an eighty twenty ratio with stratification enabled, so that both classes are represented in the same proportion in each split. This left thirty-two training samples and eight test samples.

Define dataset split

Split ratios

Define how your dataset is split across training and test.

Training
 %

Test
 %

Stratified splitting

☐ Exclude disabled samples

☐ Stratify samples by label ?

Data preview

See how your data is affected by your options

[Preview dataset split](#)

[Cancel](#)
[Create dataset split](#)

Figure 2: Stratified eighty twenty train and test split

Impulse Design

The impulse uses a time series input block with a window size of 2000 ms and a window increase of 80 ms, operating at the 100 Hz sampling frequency of the collected data. The three accelerometer axes were selected as inputs to a Spectral Analysis processing block, followed by a neural network classification block with two output classes.

The spectral analysis parameters were set using the autotune option. This selected a low-pass filter with a cutoff of approximately 13.1 Hz and order six, an input decimation ratio of three, and an FFT length of thirty-two with a logarithmic spectrum. The resulting processing block requires 11 ms of processing time and 3 KB of peak RAM on the target device.

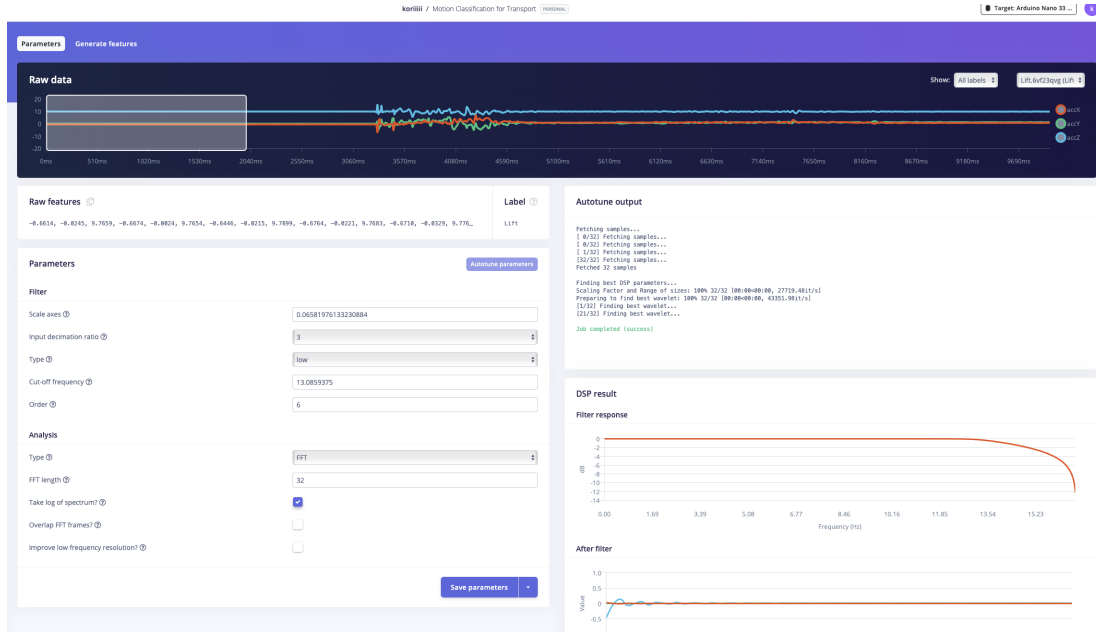


Figure 3: Autotuned spectral analysis parameters and DSP result

Generating features produced 3232 training windows across the two classes. The feature explorer shows the Idle and Lift windows forming clearly separated groups with essentially no overlap, which indicates that the spectral features carry enough information to distinguish the two states before any training has taken place.

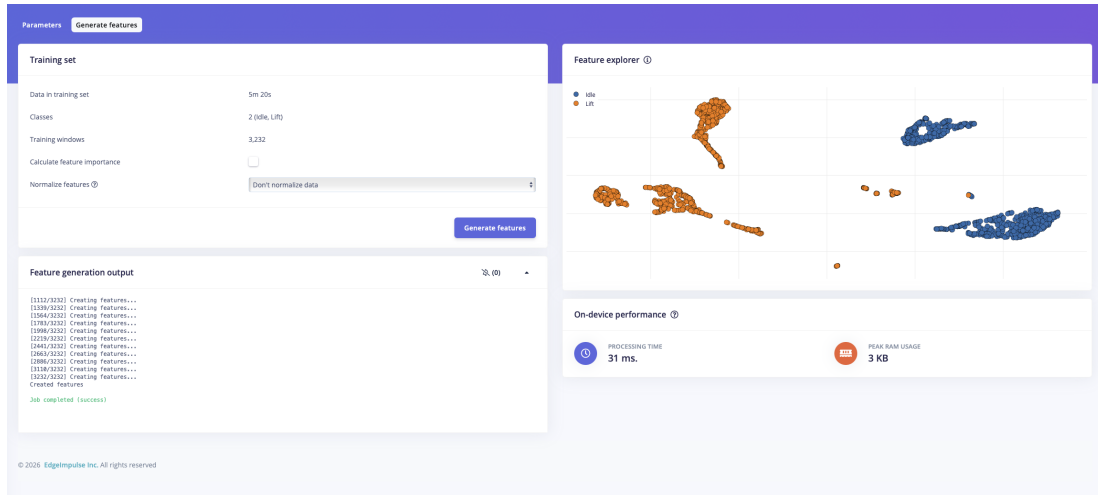


Figure 4: Feature explorer showing separation between the Idle and Lift classes

Model Training and Results

The classifier was trained for thirty cycles with a learning rate of 0.1, using a network with an input layer of fifty-four features, two hidden dense layers of twenty and ten neurons, and a two-class output layer. Training reached 100% accuracy on the validation set with a loss of 0.00, and the confusion matrix shows no misclassifications in either direction.

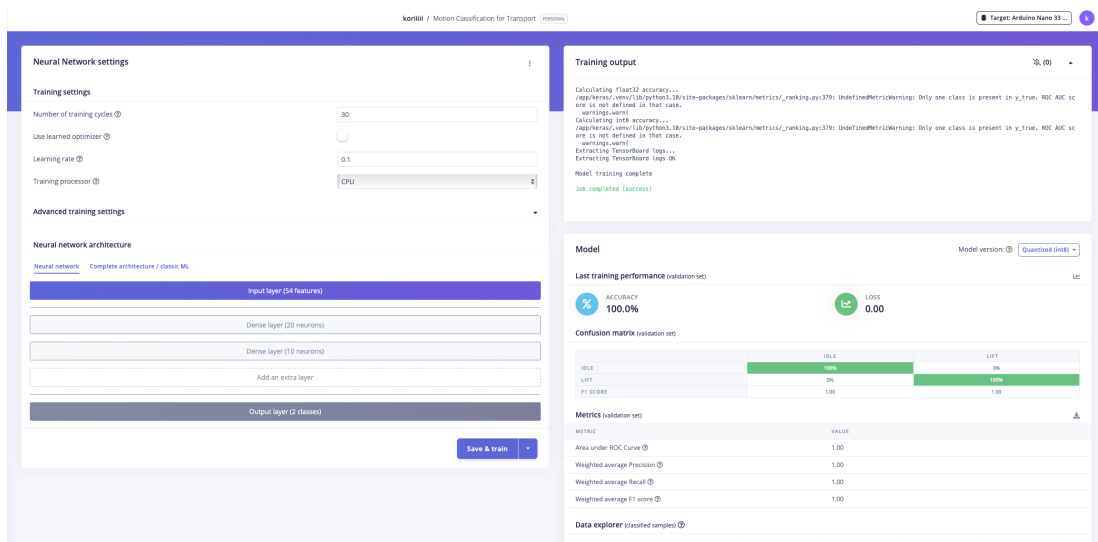


Figure 5: Neural network settings, training output and confusion matrix

The quantized int8 model requires 1 ms of inference time, 1.4 KB of peak RAM, and 15.4 KB of flash. All classified samples in the data explorer are marked correct.

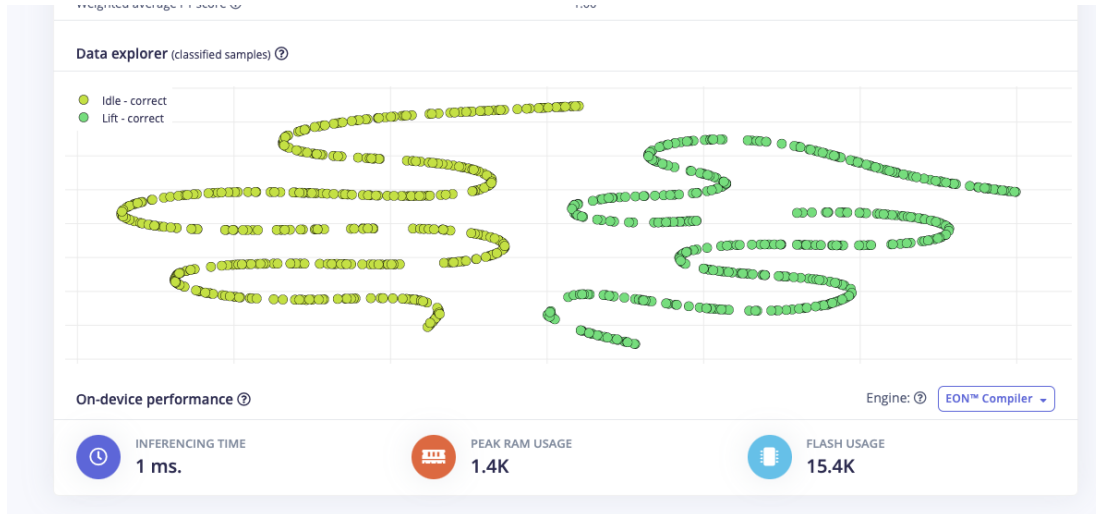


Figure 6: Classified samples and on-device performance figures

The perfect validation accuracy should be read with some caution. Because every Lift sample was recorded with the board held in a different orientation from the flat desk position used for Idle, the mean value of the accelerometer axes alone is close to sufficient for separating the two classes. The model is therefore likely relying heavily on static orientation rather than on the dynamics of the lifting motion. A Lift performed with the board kept level would probably not be recognised. This is a limitation of how the data was collected rather than a failure of the model.

Deployment and On-Device Testing

The impulse was built as an Arduino Nano 33 BLE Sense firmware binary with the EON compiler and int8 quantization enabled, then flashed onto the board using the provided flash script. Running `edge-impulse-run-impulse` produced live classification output over serial.

The deployed model correctly identifies both states. With the board resting on the desk the Idle class is predicted with a score of 0.996, and when the board is lifted the prediction switches to Lift with the same score. The reported timing is 21 ms for DSP and between 0.6 and 6 ms for inference.

```

[0.00] Started inferencing, press CTRL+C to stop...
LSE
Inferencing settings:
    Interval: 10.00 ms.
    Frame size: 600
    Sample length: 2000 ms.
    No. of classes: 2
Starting inferencing, press 'b' to break
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 21 ms, inference 6 ms, anomaly 0 ms, postprocessing 67 us
#Classification predictions:
    Idle: 0.996094
    Lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 21 ms, inference 570 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
    Idle: 0.003906
    Lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 21 ms, inference 6 ms, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
    Idle: 0.000000
    Lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 21 ms, inference 571 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
    Idle: 0.000000
    Lift: 0.996094
Starting inferencing in 2 seconds...

```

Figure 7: Live on-device classification showing both the Idle and Lift states

Part II: Tiny Ensemble Learning

Question 1: Model Architecture and Ensemble Flow

The pipeline takes an IMU window of 100 time steps and 6 features, flattened to a 600-dimensional input vector. This input is fed into three parallel branches that differ only in how the input is scaled: one branch uses the raw window, one uses a standard-scaled version, and one uses a min-max-scaled version.

Each branch begins with an encoder that was pretrained as the first half of an autoencoder. The full autoencoder maps 600 inputs through a dense layer of 64 neurons with ReLU activation to a latent layer of 32 neurons with linear activation, and then back out through a dense layer of 64 neurons with ReLU activation to a reconstruction layer of 600 neurons with linear activation. After pretraining, the decoder half is discarded and only the 600 to 32 encoder is retained.

The 32-dimensional latent vector from each encoder is passed into a classifier consisting of a dense layer of 20 neurons with ReLU activation followed by an output layer of 12 neurons with softmax activation, corresponding to the twelve activity classes.

The three branch outputs, each of size 12, are concatenated into a single vector of length 36. This vector is the input to the stacked meta-classifier, which consists of a dense layer of 24 neurons with ReLU activation followed by an output layer of 12 neurons with softmax activation. The predicted activity is the argmax of this final softmax output.

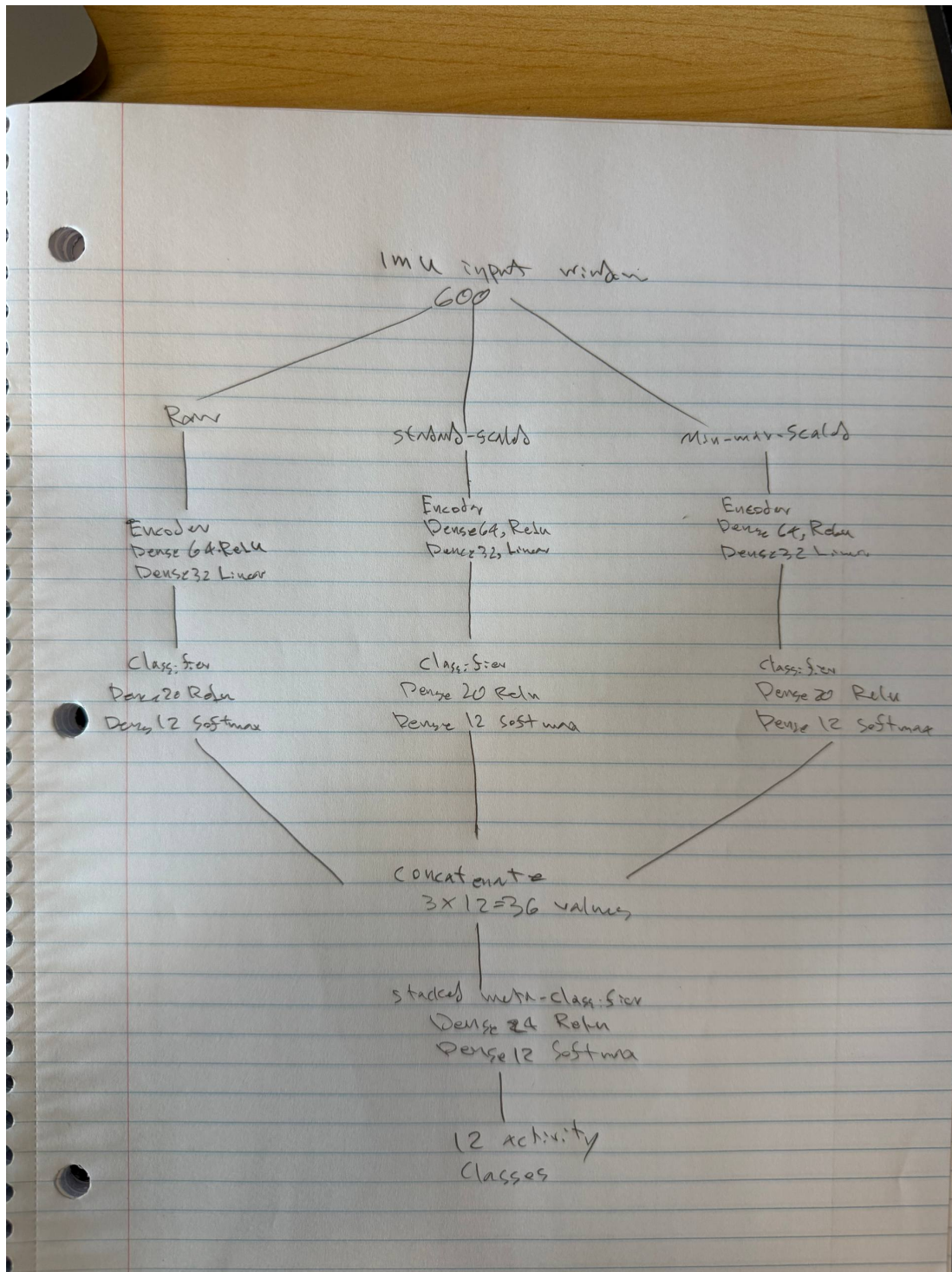


Figure 8: Ensemble architecture from input window to final prediction

For deployment, each branch is collapsed into a single sequential model that maps the 600-dimensional window directly to a 12-class softmax output through dense layers of 64 with ReLU, 32 with linear, 20 with ReLU, and 12 with softmax. This is why four model files are deployed to the Arduino rather than seven: three merged branch models and one meta-classifier.

Question 2: Pruning and QAT Methodology

Pruning is applied using magnitude-based weight pruning with a polynomial decay sparsity schedule. The schedule begins at an initial sparsity of 10% at step zero and increases to a final sparsity of 80% by the end step, so the network is sparsified gradually over training rather than having most of its weights removed at once. Gradual sparsification gives the remaining weights an opportunity to adapt and compensate as connections are removed, which preserves accuracy far better than pruning to the target level in a single step.

The pruning wrappers are stripped before quantization-aware training because those wrappers are training-time constructs. They add mask variables and update operations to each layer that exist only to implement the pruning schedule, and they are not part of the arithmetic the model performs at inference time. Quantization-aware training needs to insert fake-quantization operations into the actual inference graph, and leaving the pruning wrappers in place would mean quantizing a graph that does not match what will eventually be deployed. Stripping the wrappers returns a clean dense model whose weights happen to contain many zeros.

Quantization-aware training is then applied to this stripped model. It simulates int8 rounding during the forward pass while keeping gradients in floating point, so the network learns weights that remain accurate once they are actually quantized, rather than having quantization error introduced only after training has finished.

The additional mask-enforcement step is necessary because stripping the pruning wrappers also removes the mechanism that was keeping the pruned weights at zero. Once quantization-aware training begins, gradient updates are free to move those zeroed weights away from zero, and the sparsity achieved by pruning is quietly lost over the course of QAT. The notebook addresses this by extracting the binary masks from the stripped model, then reapplying them through a callback at the start and end of every batch and once more after training completes. This forces the previously pruned weights back to zero after each update, so the model that emerges from QAT is both quantization-ready and still 80% sparse. A simple prune-then-QAT pipeline without this step would produce a quantized model with far less sparsity than intended.

The final conversion to int8 TFLite uses full integer quantization, with the input and output tensors also set to int8. Representative data is required because quantization needs to know the actual numerical range of the activations at every layer in order to choose the scale and zero-point for each tensor. Weight ranges can be read directly from the trained model, but activation ranges depend on the data flowing through the network and cannot be determined from the weights alone. The converter therefore runs a few hundred real training windows through the model and records the observed minimum and maximum at each tensor. Using unrepresentative data here would produce badly calibrated scale factors and a large accuracy drop.

Pruning and int8 quantization matter on a device like the Arduino Nano 33 BLE Sense because that board has 1 MB of flash and 256 KB of SRAM shared across the entire application. Int8 quantization reduces the size of every weight by a factor of four compared to float32 and lets inference run using integer arithmetic, which is considerably faster on a Cortex-M4 core that has no floating-point acceleration for these operations. Pruning removes most of the weights outright, which shrinks the stored model further and reduces the number of multiply-accumulate operations performed per inference. Together they make it feasible to fit four models on the board at once and run them within the memory available for the tensor arena.

Question 3: Arduino Deployment and Real-World Behavior

The quantized ensemble was deployed to the Arduino Nano 33 BLE Sense using the provided `Tiny_Ensemble_Learning.ino` sketch together with the four generated C array files. The sketch collects 100 samples at 50 Hz to form a two-second window, applies the three scaling variants using the standard and min-max parameters exported from the notebook, runs each branch model, concatenates the three softmax outputs, and passes them to the meta-classifier. The accelerometer readings are converted from g to metres per second squared and the gyroscope readings from degrees per second to radians per second, matching the units of the training data.

With the board placed flat on a stable surface, the serial monitor reports a prediction of Standing still with a confidence score of 0.5000, repeated on every inference window. The output is not merely stable in its label; the raw, standard-scaled and min-max-scaled score vectors are numerically identical from one window to the next, which is consistent with the board receiving an essentially unchanging input while stationary.

```
14:11:46.192 -> Final prediction: Standing still
14:11:46.192 -> Class index: 0
14:11:46.192 -> Confidence score: 0.5000
14:11:46.718 -> Samples collected: 25
14:11:47.247 -> Samples collected: 50
14:11:47.774 -> Samples collected: 75
14:11:48.303 -> Samples collected: 100
14:11:48.303 -> Window collected. Running ensemble inference...
14:11:48.334 -> Raw model scores: 0.0000, 0.0000, 0.6094, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.3906
14:11:48.334 -> Standard-scaled model scores: 0.0000, 0.9961, 0.0039, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
14:11:48.334 -> Min-max-scaled model scores: 0.0000, 0.0391, 0.2891, 0.0000, 0.0000, 0.0000, 0.5352, 0.0000, 0.0000, 0.0000, 0.1367
14:11:48.367 -> Meta-classifier scores: 0.5000, 0.0977, 0.2266, 0.0039, 0.0039, 0.0039, 0.1055, 0.0039, 0.0312, 0.0117, 0.0039, 0.0156
14:11:48.367 -> Final prediction: Standing still
14:11:48.367 -> Class index: 0
14:11:48.367 -> Confidence score: 0.5000
14:11:48.894 -> Samples collected: 25
14:11:49.422 -> Samples collected: 50
14:11:49.949 -> Samples collected: 75
14:11:50.476 -> Samples collected: 100
14:11:50.476 -> Window collected. Running ensemble inference...
14:11:50.507 -> Raw model scores: 0.0000, 0.0000, 0.6094, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.3906
14:11:50.507 -> Standard-scaled model scores: 0.0000, 0.9961, 0.0039, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
14:11:50.541 -> Min-max-scaled model scores: 0.0000, 0.0391, 0.2891, 0.0000, 0.0000, 0.0000, 0.5352, 0.0000, 0.0000, 0.0000, 0.1367
14:11:50.541 -> Meta-classifier scores: 0.5000, 0.0977, 0.2266, 0.0039, 0.0039, 0.0039, 0.1055, 0.0039, 0.0312, 0.0117, 0.0039, 0.0156
14:11:50.541 -> Final prediction: Standing still
14:11:50.541 -> Class index: 0
14:11:50.541 -> Confidence score: 0.5000
14:11:51.068 -> Samples collected: 25
14:11:51.593 -> Samples collected: 50
```

Figure 9: Serial output with the board stationary on a flat surface

When the board was picked up and moved, the prediction did change, but not in a way that corresponded to the motion being performed. Successive windows produced Climbing stairs with a confidence of 0.8047 and then Lying down with a confidence of 0.6133. The labels shift as the input changes, but they do not track the actual activity.

```

40.627 -> -----
40.627 -> Final prediction: Climbing stairs
40.627 -> Class index: 4
40.627 -> Confidence score: 0.8047
40.627 -> -----
41.153 -> Samples collected: 25
41.680 -> Samples collected: 50
42.209 -> Samples collected: 75
42.737 -> Samples collected: 100
42.737 -> Window collected. Running ensemble inference...
42.768 -> Raw model scores: 0.0000, 0.0000, 0.2109, 0.0000, 0.0781, 0.0000, 0.0000, 0.0000, 0.0000, 0.0937, 0.4766, 0.1367
42.768 -> Standard-scaled model scores: 0.0039, 0.0000, 0.6094, 0.0000, 0.1914, 0.0000, 0.0000, 0.0000, 0.0000, 0.1914, 0.0000, 0.0000
42.768 -> Min-max-scaled model scores: 0.0000, 0.0703, 0.1484, 0.0000, 0.5937, 0.0000, 0.1289, 0.0000, 0.0000, 0.0000, 0.0586, 0.0000
42.801 -> Meta-classifier scores: 0.2070, 0.0039, 0.6133, 0.0000, 0.1367, 0.0000, 0.0000, 0.0000, 0.0039, 0.0195, 0.0000, 0.0117
42.801 -> -----
42.801 -> Final prediction: Lying down
42.801 -> Class index: 2
42.801 -> Confidence score: 0.6133
42.801 -> -----
43.328 -> Samples collected: 25
43.855 -> Samples collected: 50
44.383 -> Samples collected: 75
44.909 -> Samples collected: 100
44.909 -> Window collected. Running ensemble inference...
44.940 -> Raw model scores: 0.0000, 0.0000, 0.0195, 0.0000, 0.0469, 0.4141, 0.0039, 0.0000, 0.0000, 0.0117, 0.4922, 0.0117
44.940 -> Standard-scaled model scores: 0.0000, 0.0273, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.9688, 0.0000, 0.0000, 0.0000, 0.0000
44.940 -> Min-max-scaled model scores: 0.0000, 0.1055, 0.0156, 0.0039, 0.0039, 0.0000, 0.6211, 0.0000, 0.0000, 0.0000, 0.2461, 0.0000
44.973 -> Meta-classifier scores: 0.3203, 0.0547, 0.1445, 0.0000, 0.0391, 0.0039, 0.3828, 0.0000, 0.0352, 0.0117, 0.0039, 0.0078
44.973 -> -----

```

Figure 10: Serial output with the board being moved

Question 4: Deployment Behavior on the Arduino

The deployed ensemble did not behave sensibly. While the board was stationary it reported Standing still on every window at a confidence of 0.5000, and once the board was moved it began producing labels such as Climbing stairs and Lying down that bore no relationship to the motion being performed. In neither case was the prediction meaningful.

The clearest evidence of what is going wrong appears in the individual branch scores rather than in the final label. On a single stationary window, the raw branch assigned its highest score to class 2 at 0.6094, the standard-scaled branch assigned 0.9961 to class 1, and the min-max-scaled branch assigned 0.5352 to class 6. Three branches looking at the same input arrived at three different and mutually exclusive answers, one of them with very high confidence. The premise of an ensemble is that its members largely agree and that combining them cancels out individual errors, so this degree of disagreement indicates that the branches are not seeing data resembling anything they were trained on. The meta-classifier is then combining three unreliable votes and returning 0.5000 for Standing still, which is a default rather than a decision.

It is also worth noting that every score printed is a multiple of 0.0039, or one part in 256. This is the output quantization step of the int8 models, and it explains why the reported confidence takes an apparently exact value such as 0.5000. The confidence figure has less precision behind it than its four decimal places suggest.

The most likely explanation for this behaviour is a mismatch between the training data and the live sensor data. The models were trained on the mHealth dataset, which was collected from sensors worn on a subject's chest and ankles while performing whole-body activities. The Arduino sits on a desk or in a hand, so the sensor is in a completely different location relative to the body and is measuring something quite different from what the training data captured. A board resting on a table does not correspond to any of the twelve activities in the training set, so the model is being asked to classify an input that lies well outside the distribution it was trained on. In that situation a softmax output will still produce a confident-looking prediction, but the prediction is essentially arbitrary.

Axis orientation is a second contributor. The mHealth sensors had a fixed and known orientation on the body, and the sign and magnitude of each accelerometer axis reflect that placement. The Arduino's axes are aligned with the board itself, so an accelerometer axis that read close to negative

one g in the training data may read close to positive one g on the board depending on which way it is facing. The standard and min-max scaling parameters were computed from the mHealth data, so applying them to values with a different sign convention produces scaled inputs that fall in the wrong part of the input space for two of the three branches.

Unit conversion is handled correctly in this sketch, since acceleration is multiplied by 9.80665 and the gyroscope is converted to radians per second, but it is worth noting that this is a common source of the same failure mode. If the conversion were omitted, the accelerometer values would be smaller than the training data by roughly a factor of ten and every branch would receive inputs far outside its calibrated range.

Two practical changes would improve deployment reliability. The first is to collect a small amount of calibration data using the board itself, in the position and orientation it will actually be used in, and either fine-tune the models on that data or at least recompute the scaling parameters from it. This directly addresses the distribution mismatch rather than working around it. The second is to add a confidence threshold and a smoothing step in the sketch, so that a prediction is only reported when the meta-classifier score exceeds some minimum and the same class has been predicted across several consecutive windows. This would prevent the board from reporting a confident label for an input it has no basis for classifying.